Mentimeter

Haskell: Introducere

Mentimeter

Caracteristici Haskell

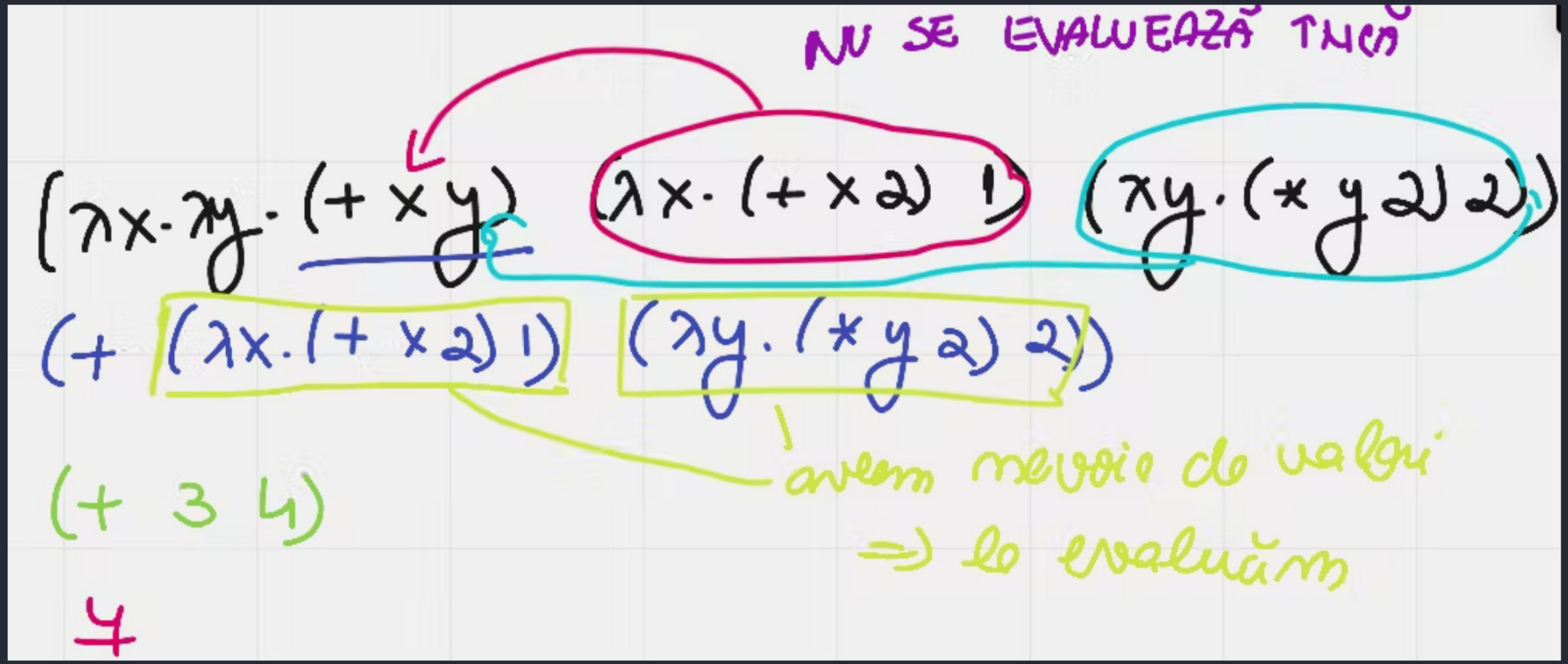
- limbaj functional **pur** => fara efecte laterale (ex: fara set ca in Racket)
- tipare statica (+ tipuri polimorfice), tare + inferenta de tipuri puternica
- evaluare lenesa (nu evaluam expresii pana nu avem nevoie de valoarea lor)
- functiile sunt implicit curry (pot fi facute si uncurry)

Mentimeter

Racket vs Haskell

- Evaluare: aplicativa vs lenesa
- Tipare: dinamica vs statica
- Efecte laterale: da vs nu
- Functii: uncurry/curry vs curry

Mentimeter



Evaluare lenesa

Mentimeter

Tipare statica, tare

- fiecare expresie are asociat un tip
- asocierea tipurilor inainte de evaluarea expresiilor
- tipare tare: nu se face conversie implicita de tip

Mentimeter

Inferenta de tipuri puternica

- fiecare functie, variabila, etc are un tip
- putem sa il declaram noi sau sa lasam compilatorul sa il determine folosind inferente
- **este recomandata scrierea tipurilor la functii pentru documentatie**

Mentimeter

ghc vs ghci

- ghc: compilare
- ghci: interpretare

Mentimeter

Folosire ghci (din terminal)

- stack exec ghci nume.hs
- (daca s-au facut modificari) :re
- (pentru a iesi din interpretor) :q

Mentimeter

Comentarii

- o singura linie: -- ceva
- mai multe linii: {- altceva -}

Mentimeter

Tipuri de baza

- Int, Char, String (echivalent cu [Char]), Bool
- se poate afisa tipul unei expresii cu :t

Mentimeter

Tipul unei functii

- functiile by default sunt curry
- ex: f :: a -> b -> c -> d
- ultimul tip e cel intors (d) si restul sunt tipul parametrilor
- (f 1) :: b -> c -> d (obtin o noua functie daca aplic partial f)

Mentimeter

```
*Main> f x y z = [x + y] : z
*Main> :t f
f :: Num a => a -> a -> [[a]] -> [[a]]
```

Inferenta de tip

Mentimeter

Ce e asta?

- prima data definim o functie f cu 3 parametri
- functia face o lista cu suma dintre x si y si o adauga la lista z
- nu am dat tipul functiei => trebuie inferat pe baza definitiei

Mentimeter

Tipul lui f 0

- din documentatie stim ca (+) :: Num a => a -> a -> a
- ce inseamna Num a? qualified type (a este un tip care apartine clasei Num)
- + e aplicat pe x, y => x, y au acelasi tip Num a => a

Mentimeter

Tipul lui f 1

- rezultatul de la x + y are tot tipul a iar lista [x+y] va avea tipul [a]
- [x+y]:z => [x+y] = element, z = lista
- toate elementele unei liste au acelasi tip, [x+y] are tipul [a] => z are tipul [[a]]
- tipul final f :: Num a => a -> a -> [[a]] -> [[a]] (ultimul tip = cel returnat = al listei facute)

Mentimeter

Liste

- constructori: [] (lista vida), (:) - ex. x:y (adauga x la inceputul listei y)
- [1, 2, 3] = 1:2:3:[]
- [a, a + b .. c] = lista tuturor elementelor din [a, c] de forma a + nb

Mentimeter

Toate elementele unei liste au obligatoriu acelasi tip (liste omogene)

Mentimeter

```haskell
list1 = [1, 5 .. 10]      --toate numerele de la 1 la 10 din 4 in 4 => [1, 5, 9]
list2 = [2, 6 .. 10]      --toate numerele de la 2 la 10 din 4 in 4 => [2, 6, 10]
head1 = head list1        --head :: [a] -> a, head1 = 1
tail2 = tail list2        --tail :: [a] -> [a], tail2 = [6, 10]
list3 = list1 ++ list2    --(++) :: [a] -> [a] -> [a], list3 = [1, 5, 9, 2, 6, 10]
is_empty1 = null list1    --null :: Foldable t => t a -> Bool, is_empty1 = False
size3 = length list3      --length :: Foldable t => t a -> Int, size3 = 6
contains0 = elem 0 list1 --elem :: (Foldable t, Eq a) => a -> t a -> Bool, contains0 = False
list4 = zip list1 list2   --zip :: [a] -> [b] -> [(a, b)], list4 = [(1, 2), (5, 6), (9, 10)]
```

Operatori pe liste



Mentimeter

```haskell
pair1 = (1, "of")                    --pair1 :: (Integer, [Char])
pair2 = ("of", ("ce", "pereche"))    --pair2 :: ([Char], ([Char], [Char]))
fst1 = fst pair1                     --fst1 = 1
snd2 = snd pair2                     --snd2 = ("ce","pereche")
```

Perechi (elementele unei perechi **nu trebuie sa aiba acelasi tip**)

1

Mentimeter

Functii

- functii anonime: \parametri_separati_prin_spatiu -> corp
- functii cu nume: nume_functie parametri_separati_prin_spatiu = corp
- ex: inc_and_div x y = (x + 1) / y sau inc_and_div = \x y -> (x + 1) / y

Mentimeter

Functii curry by default

- avem o functie f :: Char -> [[Char]] -> [[Char]]
- f primeste un caracter si un vector de string-uri (fiecare string e un vector de caractere)
- daca aplicam f pe un singur parametru obtinem o alta functie (ex: f 'a' :: [[Char]] -> [[Char]])

Mentimeter

uncurry

- putem face functiile curry (ca sa primeasca **obligatoriu** toti parametri odata)
- folosim functia uncurry :: (a -> b -> c) -> (a, b) -> c
- ex: uncurry zip :: ([a], [b]) -> [(a, b)]

Mentimeter

```haskell
numbers = [10, 23, 56, 100]
inc_numbers = map (+ 1) numbers        --map :: (a -> b) -> [a] -> [b], (+1) = \x -> x+1
odd_numbers = filter odd numbers       --filter :: (a -> Bool) -> [a] -> [a]
```

Functionale 0

Mentimeter

```haskell
digits = [1, 9, 2, 3, 5]

--foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
from_left = foldl (\acc d -> (acc * 10) + d) 0 digits

--foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
from_right = foldr (\d acc -> (acc * 10) + d) 0 digits
```

Functionale 1 (**diferenta pozitie acumulator: la foldl in stanga, la foldr in dreapta**)

Mentimeter

```haskell
nums = [10, 23, 56, 100]
inc_nums = map (+ 1) nums --[11, 24, 57, 101]

--zip :: [a] -> [b] -> [(a, b)]
--cons_pairs = [(10, 11), (23, 24), (56, 57), (100, 101)]
cons_pairs = zip nums inc_nums

--zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
--cons_lists = [[10, 11], [23, 24], [56, 57], [100, 101]]
cons_lists = zipWith (\x y -> [x, y]) nums inc_nums
```

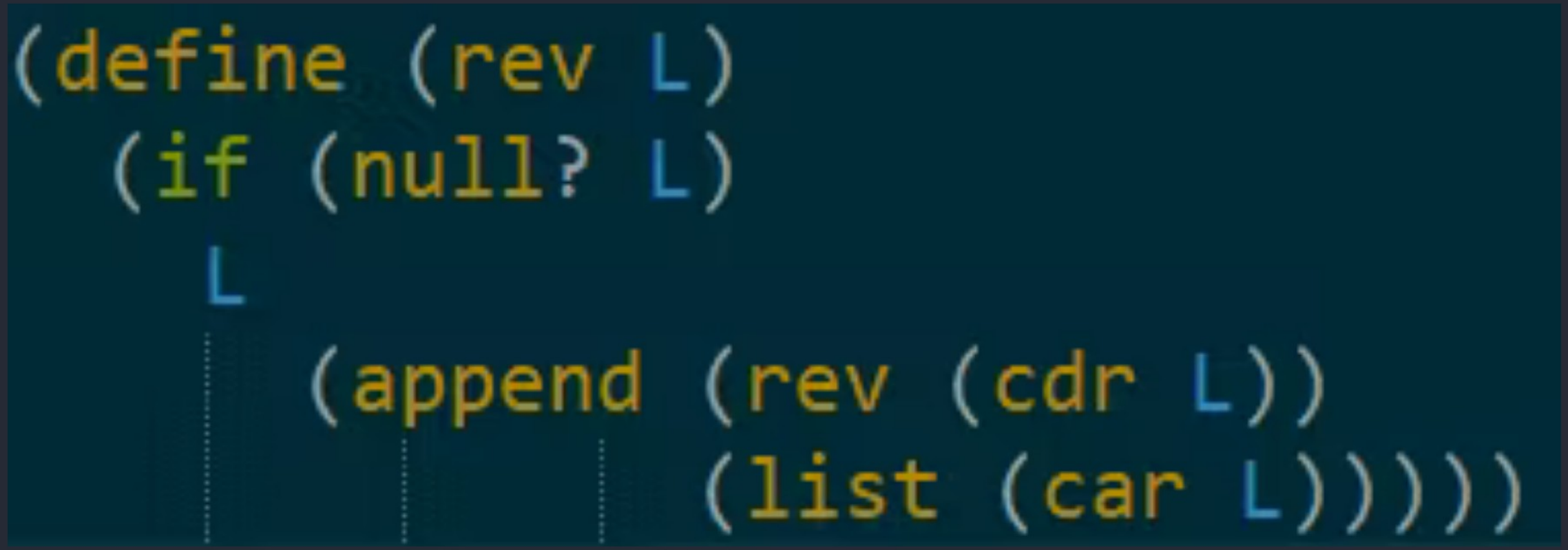
Functionale 2

Mentimeter

Definirea functiilor

- If
- Garzi
- Case
- Pattern matching

Mentimeter



Exemplu reverse list (racket)

Mentimeter

```haskell
-- if-then-else: obligatoriu ambele ramuri (then/else)
reverseList l = if null l then l else reverseList (tail l) ++ [head l]

-- pot sa am si if-uri imbricate
-- dar **obligatoriu** ce intorc pe fiecare ramura trebuie sa aiba acelasi tip
result = if False
    then "of"
    else if 2 == 2
        then "opa"
        else "nopa"
```

Solutie cu if



Mentimeter

```haskell
-- garzi: | conditie = rezultat
-- se evalueaza conditiile in ordine si la prima adevarata se intoarce valoarea si
-- **nu se mai evalueaza si restul**
-- otherwise e echivalenta cu True (se executa mereu daca ajungem pana aici)
reverseList l
    | null l = []
    | otherwise = reverseList (tail l) ++ [head l]

-- alt exemplu de garzi
-- explicatie concat: map (\c -> c:"x") w = ia fiecare litera din w si adauga x dupa ea
-- (de la un vector de caractere, obtinem un vector de string-uri => tipul = [[Char]])
-- prima garda intoarce un [Char] => **toate** celelalte garzi trebuie sa intoarca tot [Char]
-- => din [[Char]] obtinem un [Char] prin concatenarea tuturor string-urilor
transform w
    | null w = "nada"
    | length w < 6 = concat (map (\c -> c:"x") w)
    | True = "longer than 6"
new_words = map transform ["", "ceva", "prea_mult_ceva"]
```

Solutie cu garzi



Mentimeter

```haskell
-- case expresie of
--   valoare -> rezultat0 (intoarce rezultat0 daca expresie = valoare)
--   _ -> rezultat1 (nu am facut match pe nimic de pana acum => facem pe _)
reverseList l = case l of
    [] -> []
    (x : xl) -> reverseList xl ++ [x]

-- ordinea cazurilor conteaza: se evalueaza **cat timp nu s-a facut match pe un alt branch**
-- aici intra direct pe _ si intoarce "something else", chiar daca ar fi facut match si pe 4
-- apare si warning: "Pattern match is redundant"
digits = [1, 2, 3, 8]
eval = case length digits of
    0 -> "no digits"
    1 -> "only 1 digit"
    _ -> "something else"
    4 -> "[1000, 10000)"
```

Solutie folosind case



Mentimeter

```haskell
-- pattern matching: exprimam argumentele in functie de constructori
reverseList2 :: [a] -> [a]
reverseList2 [] = []
reverseList2 (x : xl) = reverseList2 xl ++ [x]

-- pot sa aplic pattern matching pe oricati parametri
-- daca nu folosesc un parametru pot sa folosesc _
-- my_zip [] l2 = [], l2 nu e folosit => my_zip [] _ = []
my_zip [] _ = []
my_zip _ [] = []
my_zip (x1 : y1) (x2 : y2) = (x1, x2) : my_zip y1 y2
```

Solutie cu pattern matching

Mentimeter

let

- asemanator letrec din Racket
- pot folosi variabile definite dupa pentru ca se vor evalua lenes **atunci cand am nevoie de ele**

Mentimeter

```
res =
    let sum = a + c             --nu se evalueaza **inca** pt ca nu stiu daca imi trebuie
        a = length [1 .. 100] --la fel si pentru restul variabilelor
        b = 1 / 0               --b nu se evalueaza inca => nu tipa de la impartirea la 0
        c = 100 ^ 2             --nici c nu e evaluat
    in sum
    --am nevoie de sum => evaluez sum => trebuie sa evalueaz si a si c
    --evalueaz a = 100, c = 10000 => sum = 10100
    --nu se genereaza nicio eroare de la b pentru ca nu ajunge sa se evalueze
```

Exemplu let

Mentimeter

where

- asemanator cu let, dar apare dupa corpul functiei
- putem folosi alte functii, variabile, etc inainte sa le definim in where

Mentimeter

```haskell
naturals = naturals_from 0
    where naturals_from i = i : naturals_from (i + 1)

-- naturals1 = naturals_from 1 genereaza eroare
-- ce definesc in where/let e vizibil numai **local**
-- odata ce am evaluat intreaga expresie nu le mai pot folosi

f value list = f new_value new_list
    where
        new_value = value * head list
        new_list = tail list
```

Exemplu where

